

Name: K. Sampath Chaitanya

Reg no: 192572006

**SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
COMPUTER SCIENCE AND ENGINEERING PROGRAMME**

Day 2: List of Programs

Question 1: Covariance, Correlation, Partitioning, and Smoothing

Aim: To calculate covariance and correlation matrices for photograph preference data, partition data into equal-frequency bins, perform smoothing by bin means and boundaries, and plot a histogram.

Algorithm:

1. Create vectors A , B , and C representing photograph preferences across age groups and assemble them into a data frame.
2. Compute sample covariance using `cov()` and correlation using `cor()`.
3. Partition a 24-element dataset into 3 equal bins of 8 values each.
4. Compute bin means using `mean()` and smooth the elements.
5. Perform boundary smoothing by comparing distances to minimum and maximum bin values.
6. Plot the frequency distribution using `hist()`.

Code:

```
R
# Part 1: Covariance & Correlation
A <- c(18, 2, 20)
B <- c(22, 28, 10)
C <- c(20, 40, 40)
preferences <- data.frame(A, B, C)

print(cov(B, C))
print(cov(preferences))
print(cor(B, C))
print(cor(preferences))
```

```
# Part 2: Equal-Frequency Binning & Smoothing
```

```
data_points <- c(18, 18, 18, 20, 20, 20, 20, 20, 20, 20, 21, 21, 21, 21, 25, 25, 25, 25, 25, 28, 28, 30, 30, 30)
```

```
bin1 <- data_points[1:8]
```

```
bin2 <- data_points[9:16]
```

```
bin3 <- data_points[17:24]
```

```
# Bin Means Smoothing
```

```
smooth_mean_bin1 <- rep(mean(bin1), length(bin1))
```

```
smooth_mean_bin2 <- rep(mean(bin2), length(bin2))
```

```
smooth_mean_bin3 <- rep(mean(bin3), length(bin3))
```

```
all_smooth_means <- c(smooth_mean_bin1, smooth_mean_bin2, smooth_mean_bin3)
```

```
print(all_smooth_means)
```

```
# Bin Boundaries Smoothing
```

```
smooth_bound_bin1 <- ifelse(bin1 - min(bin1) < max(bin1) - bin1, min(bin1), max(bin1))
```

```
smooth_bound_bin2 <- ifelse(bin2 - min(bin2) < max(bin2) - bin2, min(bin2), max(bin2))
```

```
smooth_bound_bin3 <- ifelse(bin3 - min(bin3) < max(bin3) - bin3, min(bin3), max(bin3))
```

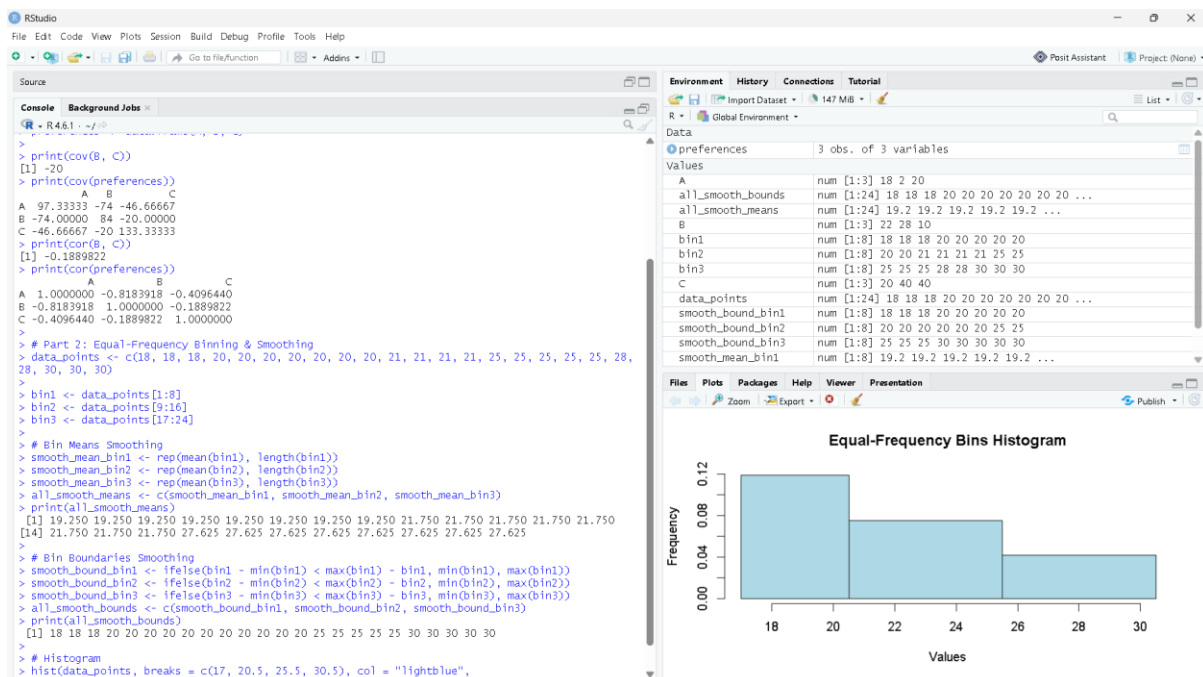
```
all_smooth_bounds <- c(smooth_bound_bin1, smooth_bound_bin2, smooth_bound_bin3)
```

```
print(all_smooth_bounds)
```

Histogram

```
hist(data_points, breaks = c(17, 20.5, 25.5, 30.5), col = "lightblue",  
     main = "Equal-Frequency Bins Histogram", xlab = "Values", ylab = "Frequency")
```

Result: Covariance and correlation matrices were generated, the dataset was smoothed via bin means and bin boundaries, and the histogram was plotted.



Question 2: Five-Number Summary of Prices

Aim: To perform a five-number summary and compute basic descriptive statistics for All Electronics prices data.

Algorithm:

1. Initialize the prices vector with the sorted price values.
2. Compute summary statistics using `summary()`, `mean()`, `median()`, and `range()`.

Code:

R

```
prices <- c(1, 1, 5, 5, 5, 5, 5, 8, 8, 10, 10, 10, 10, 12, 14, 14, 14, 15, 15, 15, 15, 15, 15, 18, 18, 18, 18, 18)
```

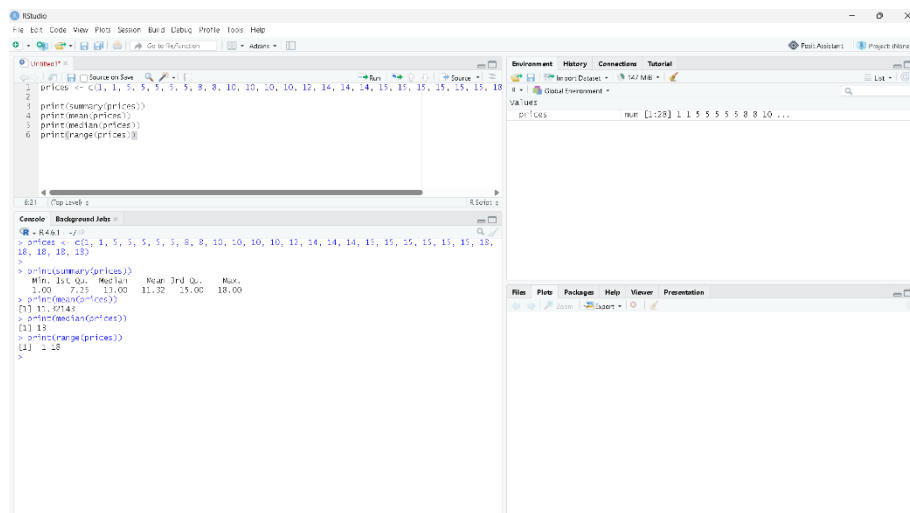
```
print(summary(prices))
```

```
print(mean(prices))
```

```
print(median(prices))
```

```
print(range(prices))
```

Result: Summary statistics, mean, median, and range were successfully evaluated for the dataset.



Question 3: Class Performance Comparison using Boxplot

Aim: To compute and compare the mean, median, and range between Class A and Class B, and visualize their score distributions with a boxplot.

Algorithm:

1. Define vectors class_A and class_B.
2. Calculate mean, median, and range (max – min) for each class.
3. Generate comparative boxplots using boxplot() with distinct fill colors.

Code:

R

```
class_A <- c(76, 35, 47, 64, 95, 66, 89, 36, 84)
```

```
class_B <- c(51, 56, 84, 60, 59, 70, 63, 66, 50)
```

```
stats_A <- c(Mean = mean(class_A), Median = median(class_A), Range = max(class_A) -  
min(class_A))
```

```
stats_B <- c(Mean = mean(class_B), Median = median(class_B), Range = max(class_B) -  
min(class_B))
```

```
print(stats_A)
```

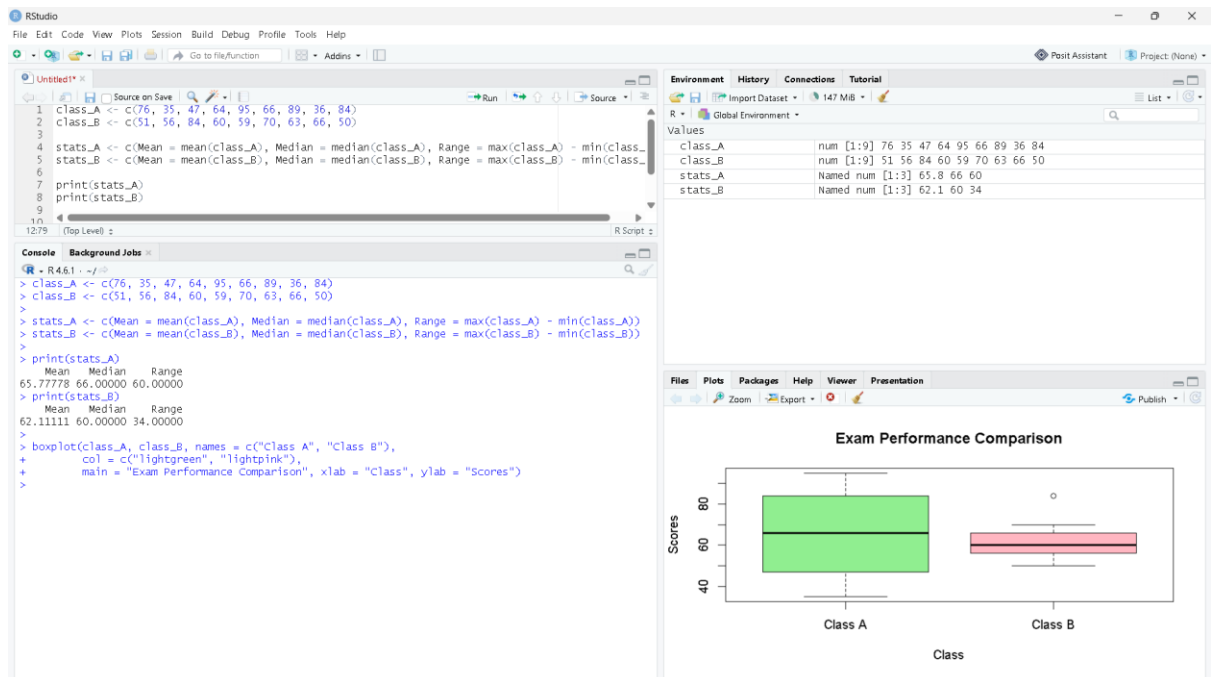
```
print(stats_B)
```

```
boxplot(class_A, class_B, names = c("Class A", "Class B"),
```

```
col = c("lightgreen", "lightpink"),
```

```
main = "Exam Performance Comparison", xlab = "Class", ylab = "Scores")
```

Result: Class A exhibits a higher mean and wider range, while Class B has a higher median and lower variance.



Question 4: Min-Max and Z-Score Normalization

Aim: To normalize a single target value into $[0,1]$ and normalize a vector of values using Min-Max and Z-Score techniques.

Algorithm:

1. Calculate Min-Max normalization for value $v = 80000$ given minimum 50000 and maximum 100000.
2. Define vector $x = [200,300,400,600,1000]$.
3. Apply Min-Max formula: $(x - \min(x)) / (\max(x) - \min(x))$.
4. Apply Z-score formula: $(x - \text{mean}(x)) / \text{sd}(x)$.

Code:

R

```
# (a) Normalizing specific value 80,000
```

```
v <- 80000
```

```
min_F <- 50000
```

```
max_F <- 100000
```

```
v_norm <- (v - min_F) / (max_F - min_F)
```

```
print(v_norm)
```

```
# (b) Normalizing dataset
```

```
x <- c(200, 300, 400, 600, 1000)
```

```
min_max_norm <- (x - min(x)) / (max(x) - min(x))
```

```
print(min_max_norm)
```

```
z_score_norm <- (x - mean(x)) / sd(x)
```

```
print(z_score_norm)
```


Result: The normalized value for 80,000 is 0.6, and the dataset is successfully scaled via Min-Max and Z-Score transformations.

The screenshot displays the RStudio interface with the following components:

- Source Editor:** Contains R code for normalizing a dataset and a specific value.
- Console:** Shows the execution output of the code.
- Environment:** Lists the objects created in the global environment.

Source Editor Code:

```
7  
8 # (b) Normalizing dataset  
9 x <- c(200, 300, 400, 600, 1000)  
10  
11 min_max_norm <- (x - min(x)) / (max(x) - min(x))  
12 print(min_max_norm)  
13  
14 z_score_norm <- (x - mean(x)) / sd(x)  
15 print(z_score_norm)
```

Console Output:

```
> # (a) Normalizing specific value 80,000  
> v <- 80000  
> min_F <- 50000  
> max_F <- 100000  
> v_norm <- (v - min_F) / (max_F - min_F)  
> print(v_norm)  
[1] 0.6  
>  
> # (b) Normalizing dataset  
> x <- c(200, 300, 400, 600, 1000)  
>  
> min_max_norm <- (x - min(x)) / (max(x) - min(x))  
> print(min_max_norm)  
[1] 0.000 0.125 0.250 0.500 1.000  
>  
> z_score_norm <- (x - mean(x)) / sd(x)  
> print(z_score_norm)  
[1] -0.9486833 -0.6324555 -0.3162278  0.3162278  1.5811388  
>
```

Environment Values:

Object	Value
max_F	1e+05
min_F	50000
min_max_norm	num [1:5] 0 0.125 0.25 0.5 1
v	80000
v_norm	0.6
x	num [1:5] 200 300 400 600 1000
z_score_norm	num [1:5] -0.949 -0.632 -0.316 0.316 1.581

Question 5: AirPassengers Custom Binned Histogram

Aim: To plot a histogram of the AirPassengers dataset starting at 100 with 150-wide intervals from 200 to 700.

Algorithm:

1. Load the built-in AirPassengers dataset.
2. Define break points: `c(100, 200, 350, 500, 650, 800)`.
3. Plot the histogram using `hist()` with custom breaks and axis points.

Code:

R

```
data(AirPassengers)
```

```
custom_breaks <- c(100, 200, 350, 500, 650, 800)
```

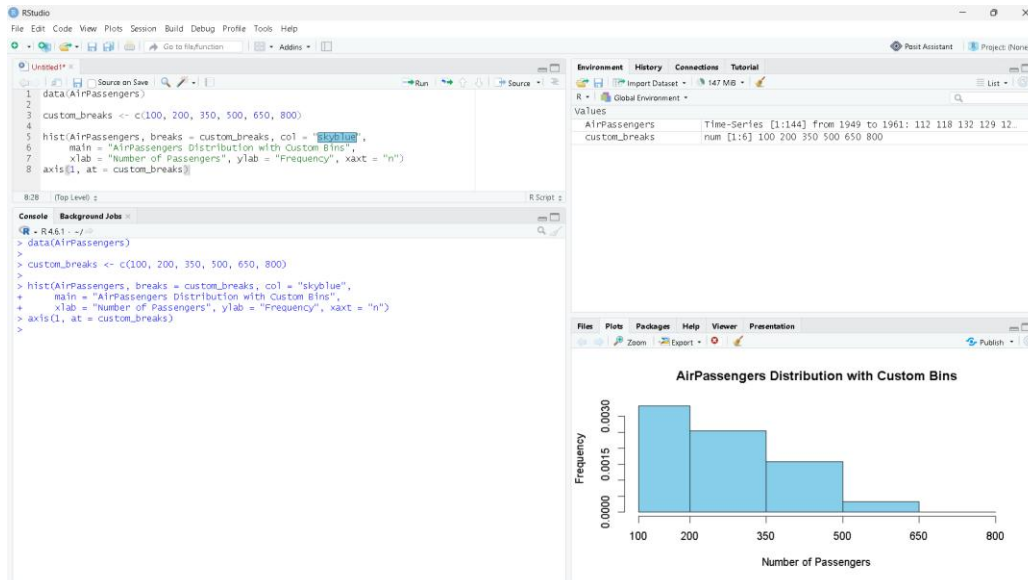
```
hist(AirPassengers, breaks = custom_breaks, col = "skyblue",
```

```
     main = "AirPassengers Distribution with Custom Bins",
```

```
     xlab = "Number of Passengers", ylab = "Frequency", xaxt = "n")
```

```
axis(1, at = custom_breaks)
```

Result: A custom-binned histogram was constructed for the AirPassengers dataset.



Question 6: Multiple Lines in a Single Line Chart

Aim: To plot multiple lines for mpg and qsec attributes of the mtcars dataset using a single plot frame.

Algorithm:

1. Load mtcars data.
2. Plot the first variable (mpg) using plot() with type = "o".
3. Overlay the second variable (qsec) using lines().
4. Add a legend to differentiate the two attributes.

Code:

R

```
data(mtcars)
```

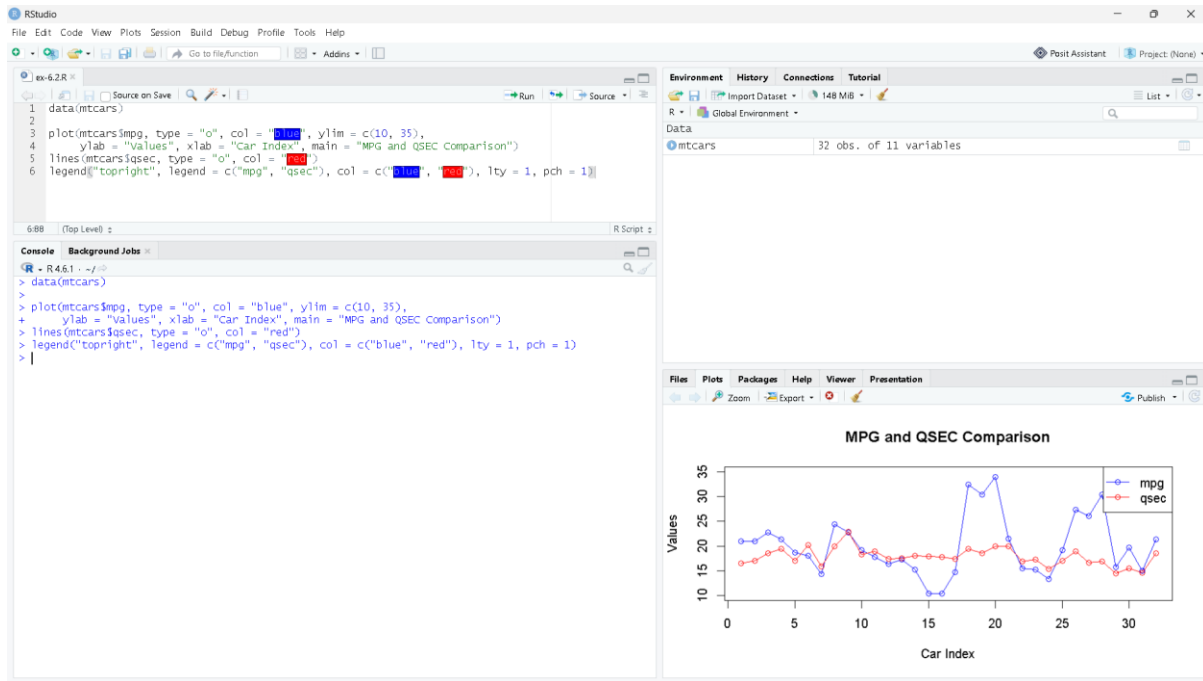
```
plot(mtcars$mpg, type = "o", col = "blue", ylim = c(10, 35),
```

```
     ylab = "Values", xlab = "Car Index", main = "MPG and QSEC Comparison")
```

```
lines(mtcars$qsec, type = "o", col = "red")
```

```
legend("topright", legend = c("mpg", "qsec"), col = c("blue", "red"), lty = 1, pch = 1)
```

Result: A multi-line chart comparing mpg and qsec across cars was generated.



Question 7: Linear Regression Model (Mortality vs Hardness)

Aim: To analyze the linear relationship between water hardness and mortality, fit a linear regression model, and predict mortality for hardness = 88.

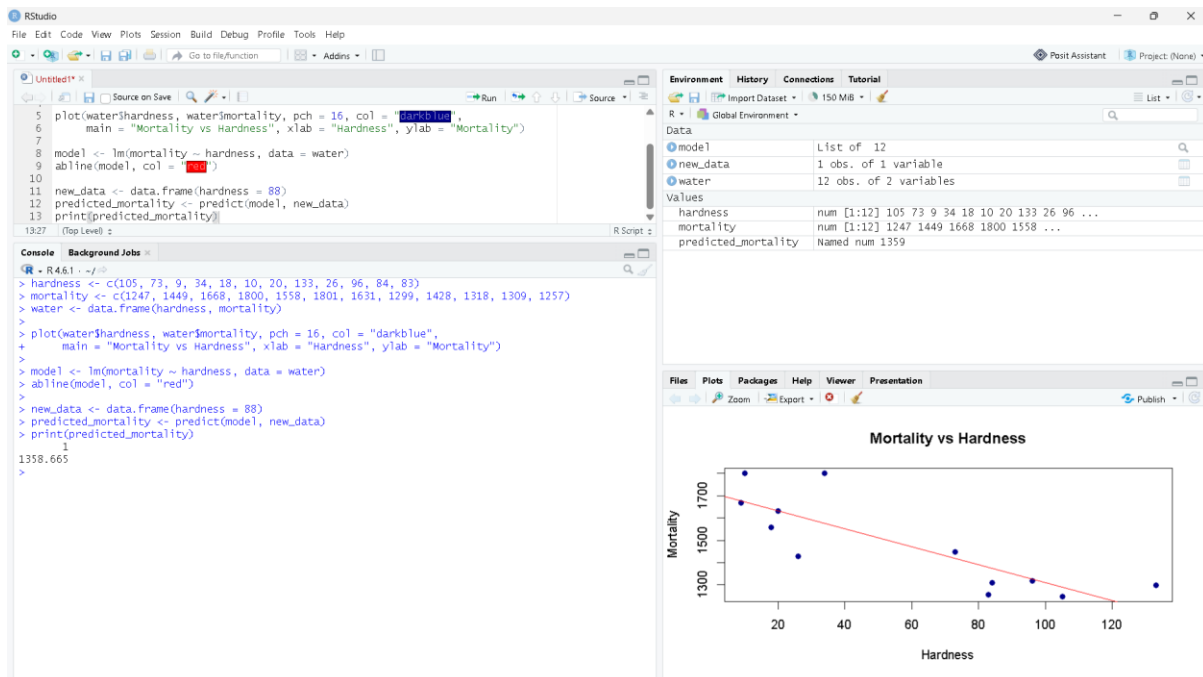
Algorithm:

1. Create the dataset attributes hardness and mortality.
2. Plot the scatter points to observe correlation.
3. Fit a linear model using `lm(mortality ~ hardness)`.
4. Add the regression line using `abline()`.
5. Predict mortality for hardness = 88 using `predict()`.

Code:

```
hardness <- c(105, 73, 9, 34, 18, 10, 20, 133, 26, 96, 84, 83)
mortality <- c(1247, 1449, 1668, 1800, 1558, 1801, 1631, 1299, 1428, 1318, 1309, 1257)
water <- data.frame(hardness, mortality)
plot(water$hardness, water$mortality, pch = 16, col = "darkblue",
     main = "Mortality vs Hardness", xlab = "Hardness", ylab = "Mortality")
model <- lm(mortality ~ hardness, data = water)
abline(model, col = "red")
new_data <- data.frame(hardness = 88)
predicted_mortality <- predict(model, new_data)
print(predicted_mortality)
```

Result: The linear model was fitted and the estimated mortality for a hardness level of 88 was computed.



Question 8: Boxplot for MPG vs Cylinders

Aim: To generate a boxplot showing the distribution of miles per gallon (mpg) grouped by the number of cylinders (cyl) in mtcars.

Algorithm:

1. Access the mtcars dataset.
2. Use `boxplot(mpg ~ cyl)` to plot grouped distributions.
3. Set appropriate colors, labels, and title.

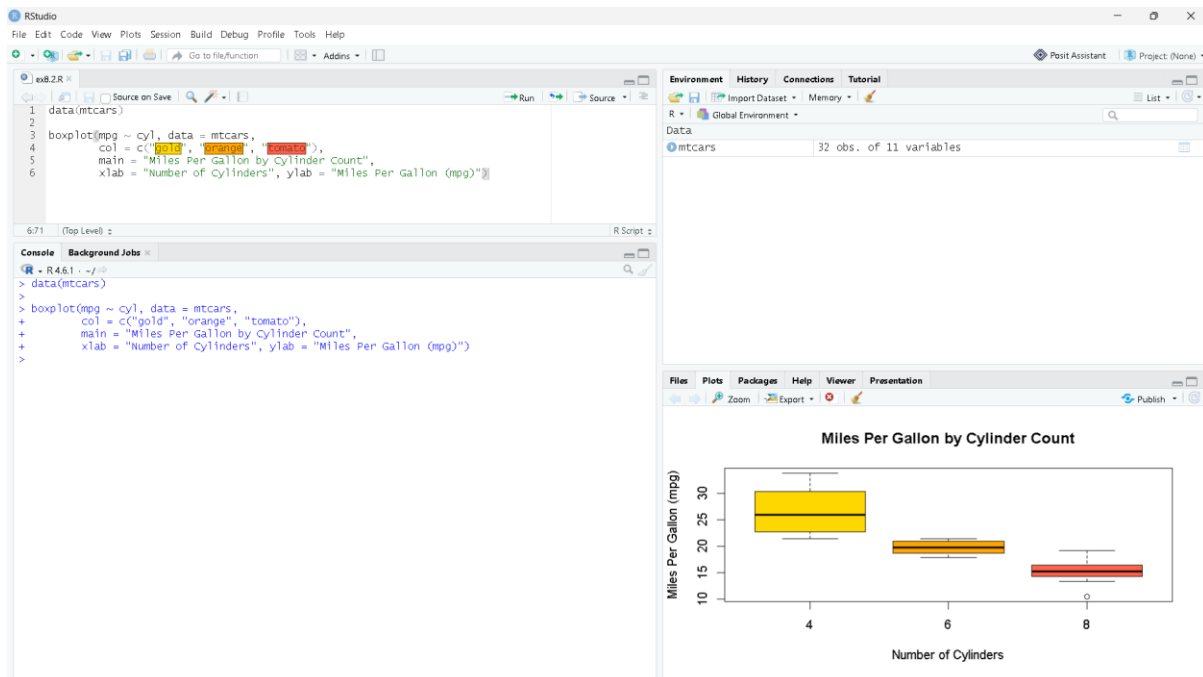
Code:

R

```
data(mtcars)
```

```
boxplot(mpg ~ cyl, data = mtcars,  
        col = c("gold", "orange", "tomato"),  
        main = "Miles Per Gallon by Cylinder Count",  
        xlab = "Number of Cylinders", ylab = "Miles Per Gallon (mpg)")
```


Result: The boxplot shows that fuel efficiency decreases as the number of cylinders increases.



Question 9: Outlier Detection using Boxplot

Aim: To demonstrate how a tennis coach can visualize scoring distributions and identify player outliers using a boxplot.

Algorithm:

1. Create a sample vector of player scores including extreme outlier values.
2. Render the boxplot using `boxplot()`.
3. Extract and display detected outliers using `boxplot.stats()$out`.

- **Code:**

R

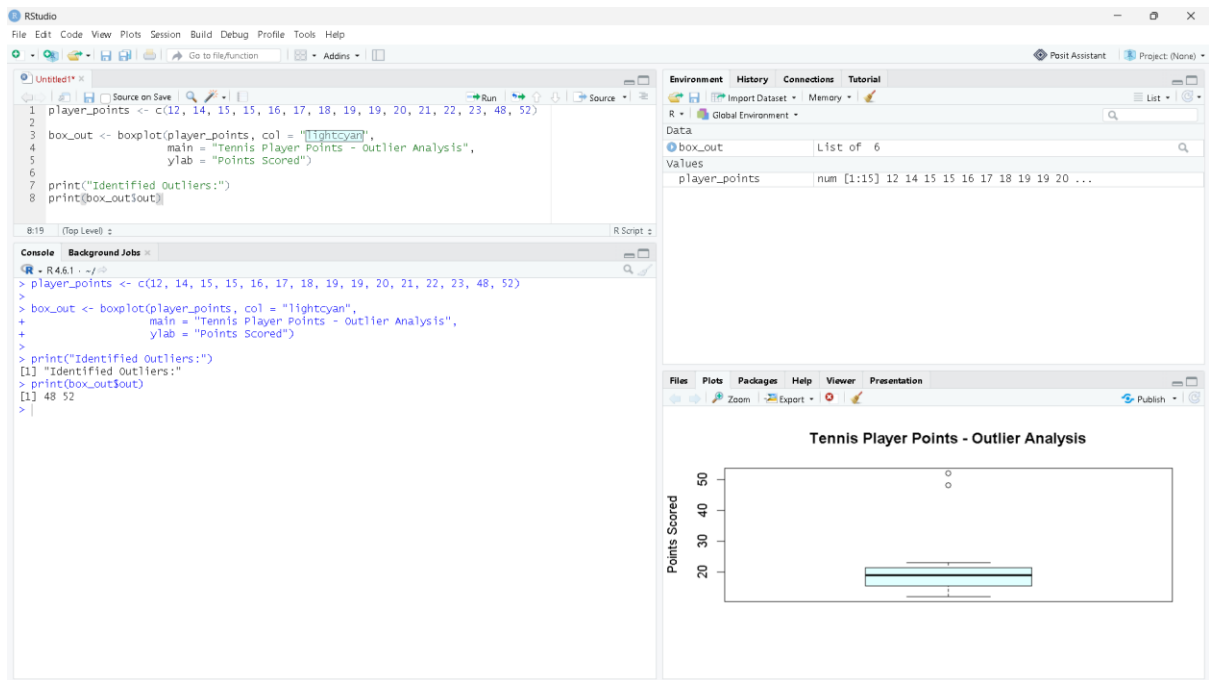
```
player_points <- c(12, 14, 15, 15, 16, 17, 18, 19, 19, 20, 21, 22, 23, 48, 52)
```

```
box_out <- boxplot(player_points, col = "lightcyan",  
  main = "Tennis Player Points - Outlier Analysis",  
  ylab = "Points Scored")
```

```
print("Identified Outliers:")
```

```
print(box_out$out)
```

- **Result:** Outliers exceeding the $1.5 \times \text{IQR}$ threshold (values 48 and 52) were detected and visually separated on the boxplot.



Question 10: Scatter Plot and Bar Chart (BloodPressure vs Age)

Aim: To analyze which age groups are affected by high blood pressure using scatter plots and bar charts from diabetes data.

Algorithm:

1. Read or sample the Age and BloodPressure attributes.
2. Create a scatter plot of BloodPressure vs Age using plot().
3. Categorize Age into discrete groups using cut() and calculate average blood pressure per group using tapply().
4. Render a bar chart of average blood pressure by age group using barplot().

Code:

R

```
age <- c(21, 22, 25, 30, 31, 35, 40, 42, 45, 50, 52, 55, 60, 65, 70)
blood_pressure <- c(60, 64, 66, 72, 70, 78, 80, 82, 85, 88, 90, 92, 95, 96, 98)
diabetes_sample <- data.frame(Age = age, BloodPressure = blood_pressure)

# Scatter Plot
plot(diabetes_sample$Age, diabetes_sample$BloodPressure,
     pch = 19, col = "purple",
     main = "Blood Pressure vs Age (Scatter Plot)",
     xlab = "Age", ylab = "Blood Pressure")
```

```
# Bar Chart by Age Group
```

```
diabetes_sample$AgeGroup <- cut(diabetes_sample$Age,  
                                breaks = c(20, 35, 50, 75),  
                                labels = c("Young (21-35)", "Middle-Aged (36-50)", "Senior (51-70)"))
```

```
avg_bp <- tapply(diabetes_sample$BloodPressure, diabetes_sample$AgeGroup, mean)
```

```
barplot(avg_bp, col = c("lightgreen", "khaki", "salmon"),  
        main = "Average Blood Pressure by Age Group",  
        xlab = "Age Group", ylab = "Average Blood Pressure")
```

Result: Scatter plot and bar chart visualizations demonstrate that blood pressure tends to increase systematically with age.

